Multitask Shape Classification

Dataset

- Split: 9,000 train / 1,000 val, 28×28 binary images, six shape types; counts per image sum to 10.
- Image example:

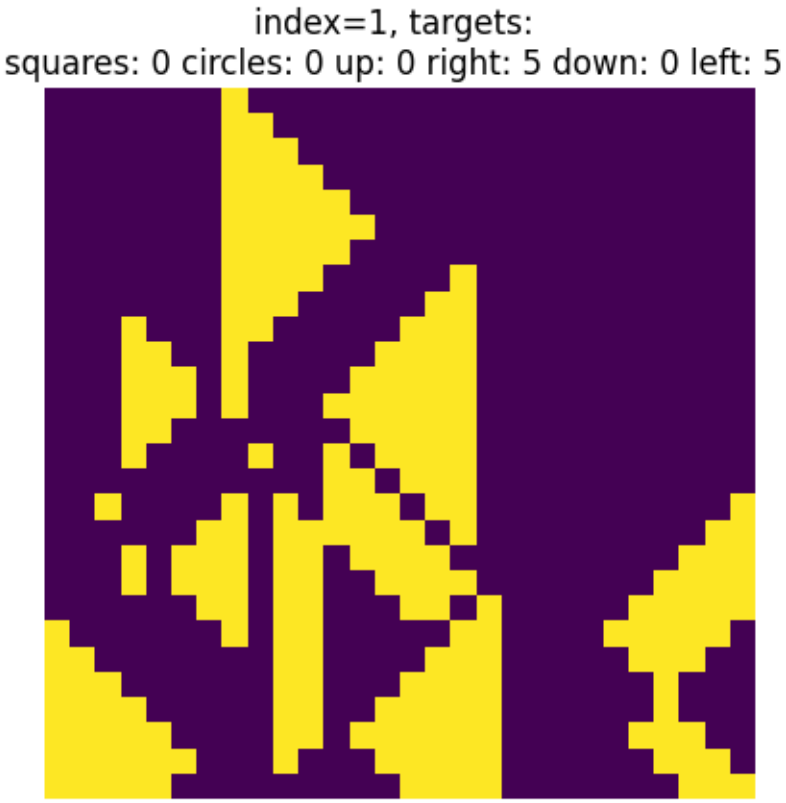


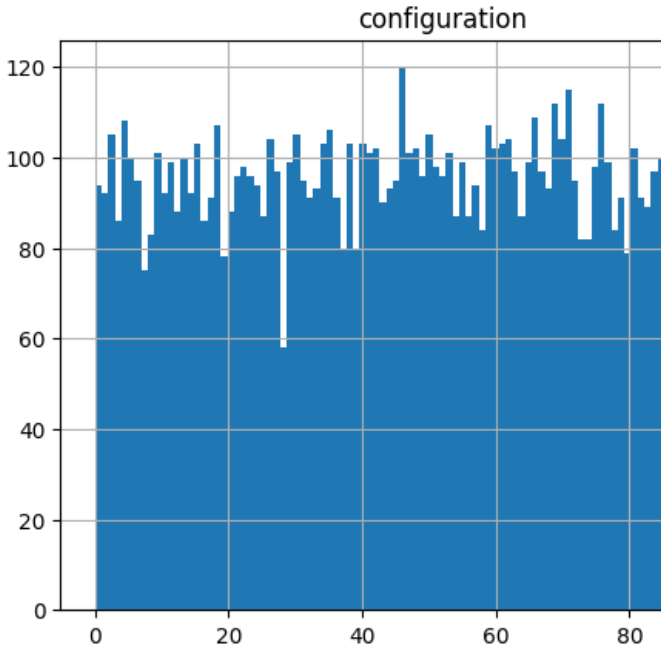
Figure 1: repr

The images are quite controvertial and sometimes really hard to get correctly (even as a human). I would argue that the labels are not correct a lot of times. - Target counts: The counts sum up to 10, and are said to be in 1..9 in the description of the assignment. However, they are actually in the range 2..8, which mean we can reduce the number of classes to 105 instead of 135. In the code, we assume that the counts are from 2 to 8, however this can be easily changed by adjusting to variables in the solution notebook.

- Shape count:

stat	squares	circles	up	right	down	left
mean	1.6574	1.7149	1.6530	1.6770	1.6857	1.6120
std	2.6291	2.6561	2.6196	2.6275	2.6411	2.5750
min	0.0	0.0	0.0	0.0	0.0	0.0
p25	0.0	0.0	0.0	0.0	0.0	0.0
median	0.0	0.0	0.0	0.0	0.0	0.0
p75	3.0	3.0	3.0	3.0	3.0	3.0
max	8.0	8.0	8.0	8.0	8.0	8.0

- Configuration distribution: We should verify that all of the classes are (almost) equally represented in the training data:



We can see that it is the case for the entire dataset, and assuming that the train/test split was done randomly, it should hold for the train/test subsets as well.

Model architecture

- Backbone fixed as provided.
- I didn't spend time looking for the best possible architecture for the heads, so they are quite shallow, while allowing to achieve 50% accuracy.

```
self.head_cls = nn.Sequential(
    nn.Linear(256, 256),
    nn.ReLU(),
```

```
            nn.Dropout(p=dropout_p),
            nn.Linear(256, num_classes),
            nn.LogSoftmax(dim=1),
        )

        self.head_cnt = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, num_counts),
        )
```

The reason head_cnt has a smaller dropout probability is that it's already very hard to train and i didn't want it to be significant.

Data Augmentation

- I've implemented all of the augmentations except for erasement, as it doesn't really make sense in my opinion - the data is already confusing enough for the model. I apply the rotations/flips with a 50% probability, as there is no reason to pick any other probability - rotation/flip doesn't impact performance while increasing the amount of different training samples.
- Contrast/Noise are applied with lower probabilities (30-40%) as they might corrupt the data sometimes, and overtraining wasn't too big of an issue to make me have to increase those.
- The implementation of those uses predefined torch transformations along with swapping of target counts if flipping/rotating. The user specifies a config and a class that composes many transformations at once is created based on it, then passed to the dataset.

Experiments

- **Exp A (cls-only)**: _cnt=0, classification loss only.
- **Exp B (reg-only)**: _cnt=1, classification loss omitted (head still evaluated).
- **Exp B.2 (multitask)**: _cnt=1, continue training the model from B but with multitasking
- **Exp C (multitask)**: _cnt=1, both losses active (best run).

Results

Experiment Results

Exp. A:

```
[Loss] loss_total=4.6562, loss_cls=4.6562, loss_reg=1.5000;
[Classification] top1=0.0110, macro_f1=0.0002, lowest_acc_pairs={'squares+up': 0.0, 'squares
[Regression] rmse_overall=3.1087, mae_overall=1.6667, rmse_per_dim={'squares': 3.0966, 'circ
```

Exp. B:

```
[Loss] loss_total=0.1451, loss_cls=4.6540, loss_reg=0.1451;
[Classification] top1=0.0110, macro_f1=0.0002, lowest_acc_pairs={'squares+up': 0.0, 'squares
[Regression] rmse_overall=0.5854, mae_overall=0.3169, rmse_per_dim={'squares': 0.4799, 'circ
```

Exp. B.2:

```
[Loss] loss_total=1.4216, loss_cls=1.2905, loss_reg=0.1312;
[Classification] top1=0.4950, macro_f1=0.4636, lowest_acc_pairs={'squares+up': 0.9143, 'squa
[Regression] rmse_overall=0.5514, mae_overall=0.3025, rmse_per_dim={'squares': 0.4646, 'circ
```

Exp. C:

```
[Loss] loss_total=1.3758, loss_cls=1.2271, loss_reg=0.1487;
[Classification] top1=0.5110, macro_f1=0.4827, lowest_acc_pairs={'right+down': 0.9032, 'up+l
[Regression] rmse_overall=0.5938, mae_overall=0.3348, rmse_per_dim={'squares': 0.4989, 'circ
```

Per-shape regression metrics (Exp C, best model)

shape	RMSE	MAE
squares	0.5882	0.3178
circles	0.5983	0.3414
triangle up	0.6473	0.3699
triangle right	0.6758	0.3674
triangle down	0.6049	0.3499
triangle left	0.6901	0.3780

Worst-paired accuracies (Exp C) remained 0.87;

Visual Results

Learning curves and confusion matrices for each experiment: - Exp A :(cls-only):

- Exp B (reg-only):

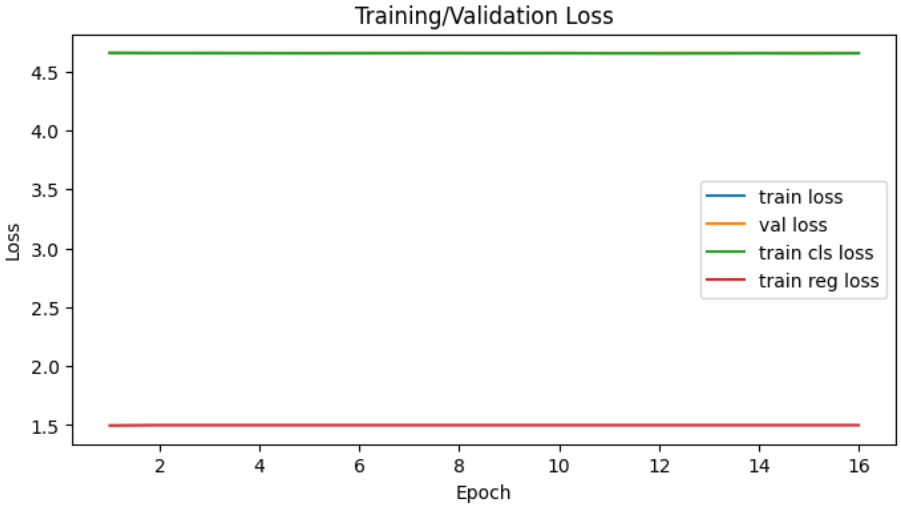
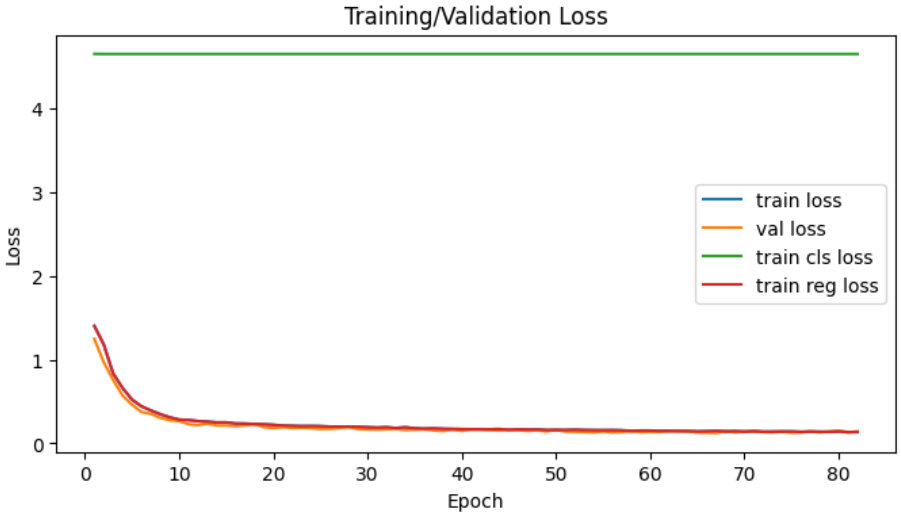
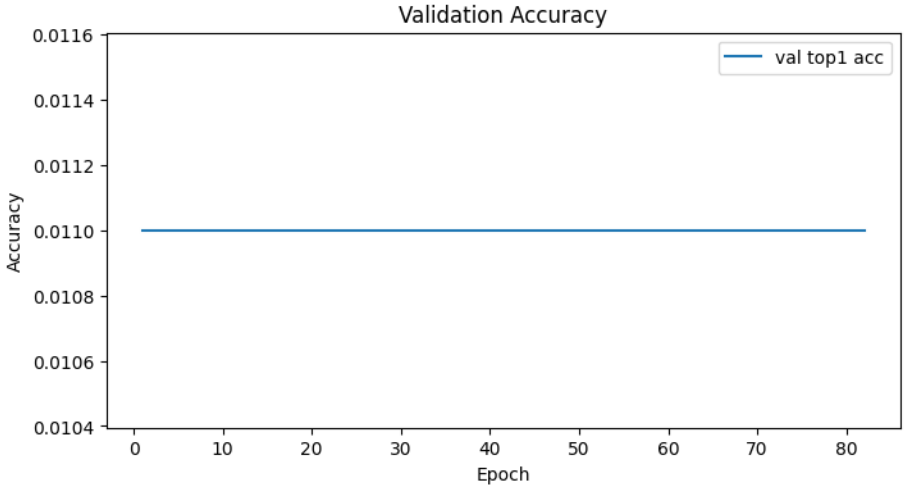
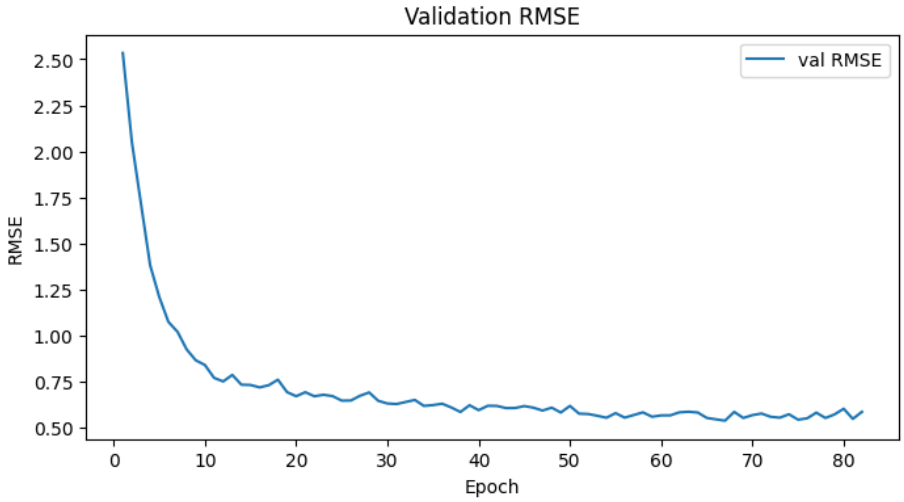


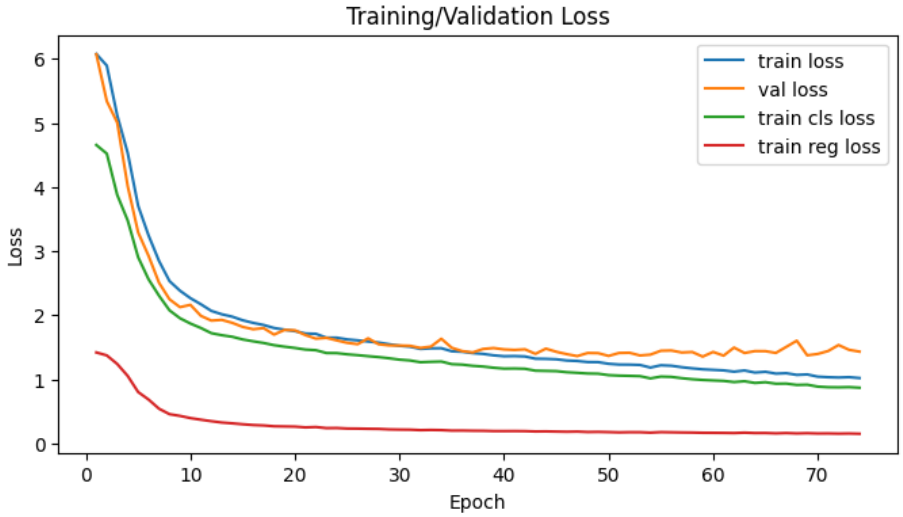
Figure 2: Loss 0 cls

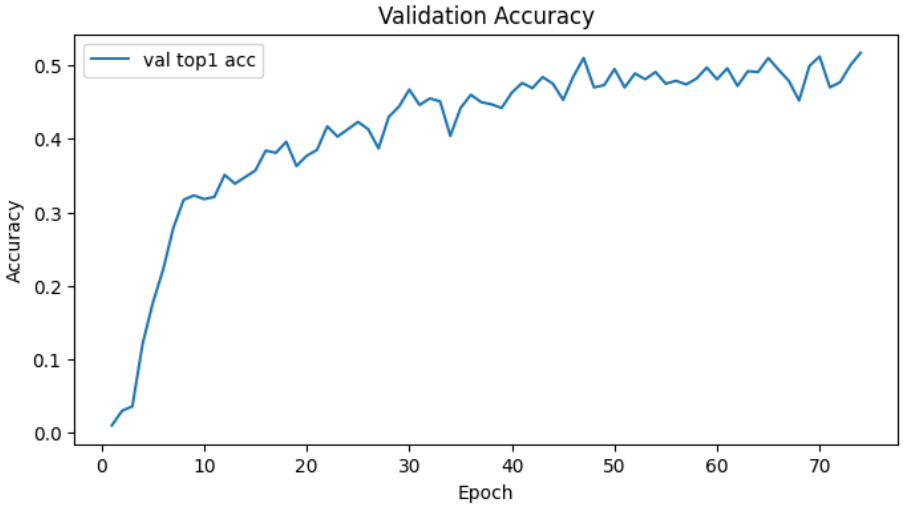


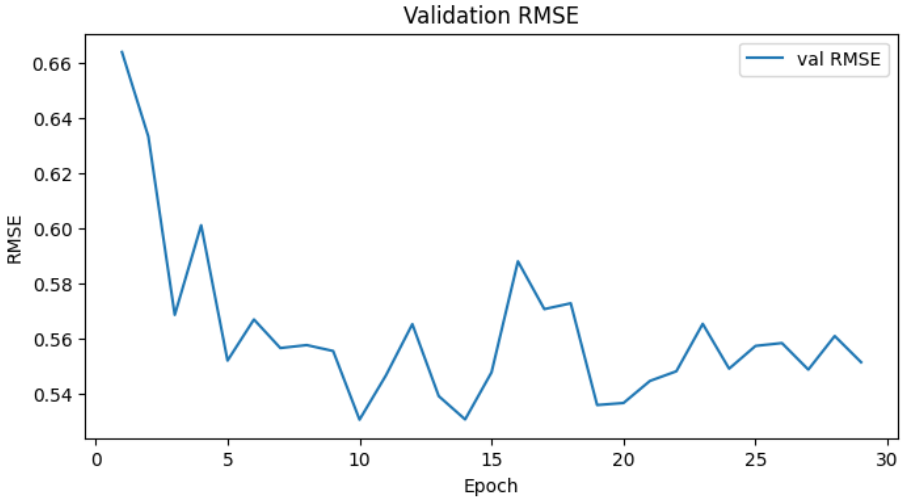




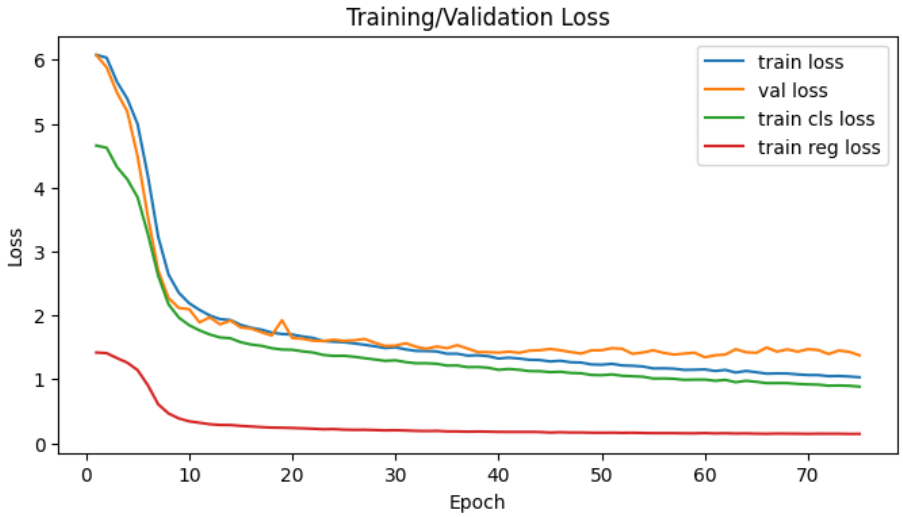
- Exp B.2 (multitask):

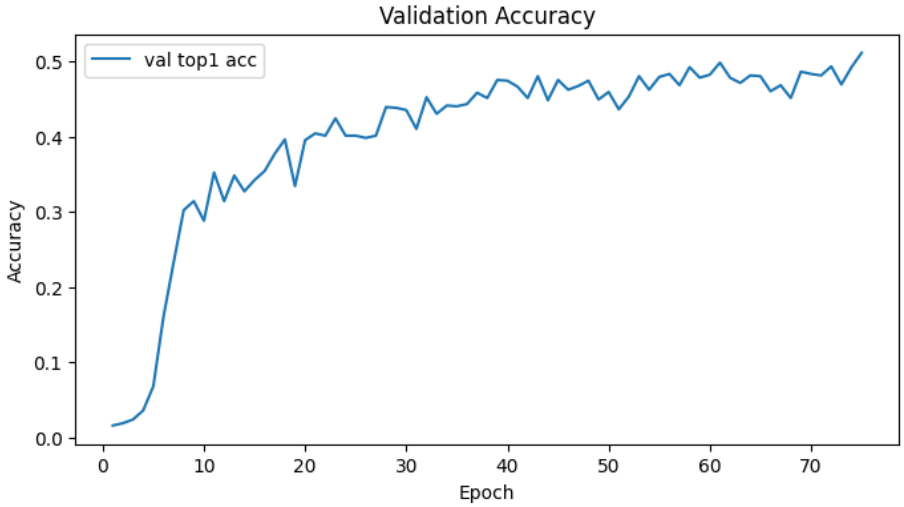

Training/Validation Loss
train loss
val loss
train cls loss
train reg loss
Loss
Epoch
0
1
2
3
4
5
6
0
10
20
30
40
50
60
70

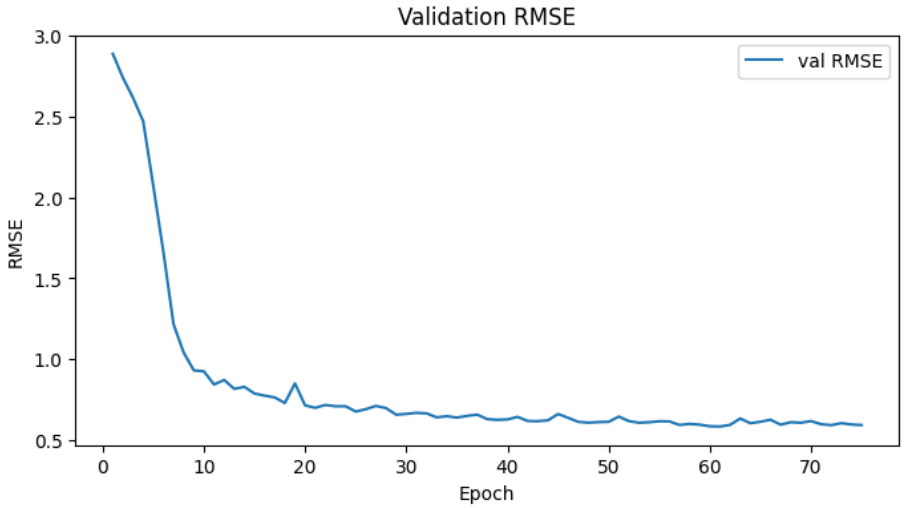

Validation Accuracy
val top1 acc
Accuracy
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0
10
20
30
40
50
60
70

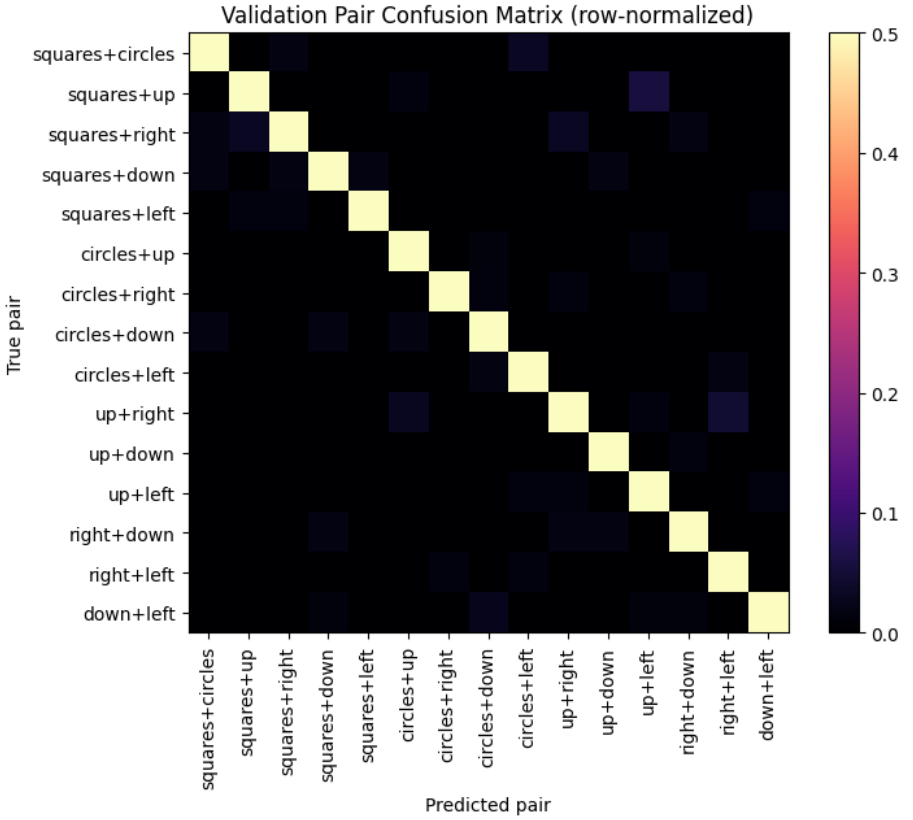


- Exp C (multitask):




Validation Accuracy
val top1 acc
Accuracy
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0
10
20
30
40
50
60
70


Validation RMSE
val RMSE
RMSE
Epoch
0.5
1.0
1.5
2.0
2.5
3.0
0
10
20
30
40
50
60
70



Discussion

I find the results conclusive: - Without multitasking, the model is nearly not training, which is to be expected considering we have >100 binary variables, and our loss does not capture the relationship between those variables: Every class has classes that are extremely similar to itself, like: 5 circles 5 squares and 4 circles 6 squares, etc. In the classification loss we do not account for the fact that the model might be off by one in counting, and still punish the model heavily. We simply can't even start training the model in this setting. - The regression works well by itself, and doesn't really need classification. - When combined, the regression part allows for much easier training of the classification part. - The best result was achieved when combining the only-regression loss with the multiclass clasification afterwards. - What's interesting is that MRSE stayed above 0.5 the entire time, which would imply that the accuracy gotten from rounding the regression outputs wouldn't be better if not worse than the classification results. - Overall, multitasking seems to be a great idea: the backbone learns more general patterns that are suitable for different tasks, which makes it easier to train the model - The regression task seems to be getting a

little bit of negative influence from the classification, but the other way around it works well.

Per class results observation (because I dont know what to do with them): - Some shape pairs were most confusing than others, squares + up were confused with up+down. - Triangle left and triangle right were the hardest to regress.